Solutions and Debrief

Stop. Finish the assessment first. Reading this before your timer runs out throws away the only thing it measures.

Working code is in `solutions/` . Grade it with `.\run.ps1 -Reference` (or `./run.sh --reference`). A printable copy of this debrief is in `Assessment-Solutions.pdf` .

Task 1 — Run-Length Compression

Approach. One pass with two indices: `i` at the start of a run, `j` scanning to the first different character. Append the character, then the length only when it exceeds 1.

Complexity. O(n) time, O(n) space for the output.

Where people lose points

- Appending `1` for single characters. The spec says bare character.
- Building the result with `String +=` in the loop — O(n²), and the 100k check catches it. Use `StringBuilder` .
- Dropping the final run by only flushing inside the loop on a character change.
- `null` input. The grader passes it.
- Assuming run lengths are single digits. `"z12"` has a two-digit count, and `charAt` arithmetic tricks break on it.

Task 2 — Department Averages

Approach. One pass to accumulate a **running total and a count** per department in a `HashMap`, then one sort. The averages are computed once, at the end, from exact numbers.

Complexity. O(R + D log D) for R records and D departments.

Where people lose points

- Keeping a running *average* instead of a total and a count. Averaging averages is wrong as soon as departments differ in size, and it compounds the rounding error at every step.
- `int` for the total. 1000 employees at 2,000,000,000 overflows long before you divide. Accumulate in `long`. The grader has a check for exactly this.
- Rounding the division. `21 / 2` must be `10`; integer division already does what the spec asks, so do not reach for `Math.round`.
- `split(",")` without a limit. Java's one-argument `split` drops trailing empty strings, so `"a,x,"` yields a length-2 array and `"a,,"` yields length 1 — both silently reshape your validation. Use `split(",", -1)`.
- Validating by `try { Integer.parseInt } catch` but forgetting that a negative number parses fine. `-5` is a valid `int` and an invalid salary.
- Forgetting the name tie-break, or writing it backwards: average descending but name **ascending**.
- Returning `null` instead of an empty array, or not clamping `n` to the number of departments.

Task 3 — Minesweeper Reveal

Approach. Three cases in order: mine clicked → `'X'` ; cell already revealed or out of bounds → return unchanged; otherwise flood fill. In the fill, count the 8 neighbours' mines first. A positive count writes the digit and stops expanding from that cell — this is what keeps the numbered ring around a mine intact. A zero count writes `'B'` and pushes the still-hidden empty neighbours.

Complexity. O(rows · cols), with each cell entering the queue at most once.

Where people lose points

- Recursive DFS. A 300×300 all-empty board is 90,000 deep and throws `StackOverflowError` ; the grader has a check for exactly that. Use an explicit `ArrayDeque` .
- Expanding from numbered cells, which floods the whole board.
- Enqueueing a cell more than once. Mark it as queued when you push it, not when you pop it, or the frontier grows exponentially on large empty boards.
- Revealing mines during the cascade. Only a direct click does that.
- Returning a new array instead of mutating in place — the grader checks identity.
- Missing the "already revealed" guard, so a second click on a `'B'` recounts and corrupts the board.

Task 4 — Longest Balanced Segment

Approach. Map the array to a running balance: `+1` for odd, `-1` for even. A subarray `(j+1 .. i)` has equal counts exactly when `balance[i] == balance[j]` . So walk once, and for each balance value remember only the **earliest** index where it occurred — the earliest index gives the longest span. Seed the map with `balance 0 -> index -1` so a prefix that is itself balanced is measured correctly.

Complexity. O(n) time, O(n) space.

Where people lose points

- The nested-loop version is correct and times out at n = 200,000.
- Overwriting the stored index each time a balance repeats, instead of keeping the first one. You then measure the shortest span, not the longest.
- Forgetting the `0 -> -1` seed, which loses every answer that starts at index 0.
- `nums[i] % 2 == 1` is **false** for `-3` , because Java's `%` keeps the sign of the dividend. Use `nums[i] % 2 != 0` , or `(nums[i] & 1) != 0` .
- Treating `0` as odd.

This same prefix-balance trick solves "longest subarray with equal 0s and 1s", "subarray sums divisible by k", and most "equal counts of two categories" questions. It is worth having in your fingers.

Part B — CloudStorage

The design question at Level 1 decides how much pain Levels 3 and 4 cause.

The data model that survives all four levels

```
files:  name -> { size, owner }        one flat map, no directory tree
users:  id   -> { capacity, used, snapshot }
```

Four decisions do the work:

1. **Keep one flat file index.** The names look like paths but nothing in the spec needs a tree. A flat `HashMap` keeps Level 1 at O(1) and makes Level 2 a filter plus a sort.
2. **Store `used` on the user and maintain it incrementally.** Every operation that moves bytes — add, delete, merge, restore — adjusts one counter. If you instead recompute a user's usage by scanning all files, Level 3 still passes and the code gets slower and easier to get wrong at Level 4.
3. **Put the ownership on the file, not a file list on the user.** Then `mergeUser` is a relabel, and no two structures can disagree about who owns what. The cost is that "all files of user X" is a scan — fine at this scale, and if it were not, you would add an `owner -> Set<name>` index *alongside* the authoritative field.
4. **Funnel every create and every destroy through one private method each.** In the reference that is `addEntry(owner, name, size)` and `removeEntry(name)` ; each maintains the file index and the owner's `used` counter together, so the invariant "a file's size is counted in exactly its owner's usage" cannot be broken by a caller. The nine public methods become thin wrappers.

Level-by-level notes

- **L1.** `addFile` is just `addFileBy("admin", ...)` . Writing it that way at Level 3 instead of duplicating logic is the refactor the format is testing.
- **L2.** Sort by `size` descending, then `name` ascending. The tie-break is the check people skip. Return an empty array, never `null` , and handle `n <= 0` .
- **L3.** `mergeUser` **adds** both capacities and both usages. Guard the three null cases (missing user, same id, `admin` as the source) *before* mutating anything — a half-applied merge is unrecoverable. Admin's unlimited capacity is best modelled as a flag, not as `Integer.MAX_VALUE` arithmetic, so sums cannot overflow.
- **L4.** `restoreUser` is "delete all of mine, then re-add from the snapshot", and re-adding through `addFileBy` gets the skip-if-taken rule and the capacity accounting for free. Keep `snapshot == null` ("never backed up") distinct from an empty snapshot — both end up deleting the user's files, but only one is a restore of zero files. Because the snapshot lives on the user object, `mergeUser` removing the user discards it with no extra code.

Where people lose points

- Capacity drift: `deleteFile` that forgets to credit the owner, or `mergeUser` that moves files without moving the usage.
- Modelling directories. Nothing asks for it, and it costs 20 minutes.
- Rewriting Level 1's storage at Level 3 instead of extending it. Budget for a refactor, but design so it is a small one.
- Returning `0` where the spec says `null` . `0` is a real remaining capacity.
- Mutating a map while iterating it (`ConcurrentModificationException`) in `mergeUser` / `restoreUser` . Collect the keys first, then act.

If you want more

`Revision-Book.pdf` in this folder is the companion drill set: 60 LeetCode-style problems with full Java solutions, graded easy to hard, including the two problems that were cut from this assessment for being above the bar — a layered Dijkstra and a fifth CloudStorage level with expiring files. Work the book first, sit this assessment second.